

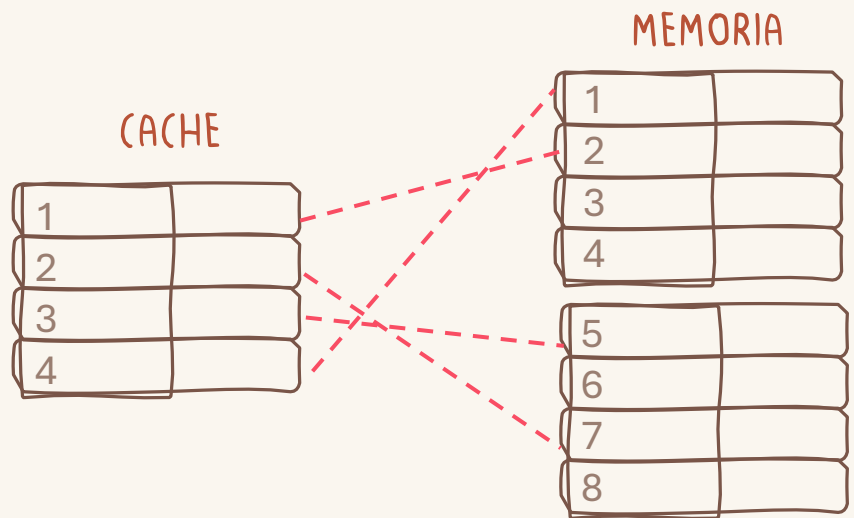
COME ASSOCIARE UN BLOCCO IN MEMORIA AD UNA POSIZIONE IN CACHE

METODI DI INDIRIZZAMENTO

ASSOCIATIVO

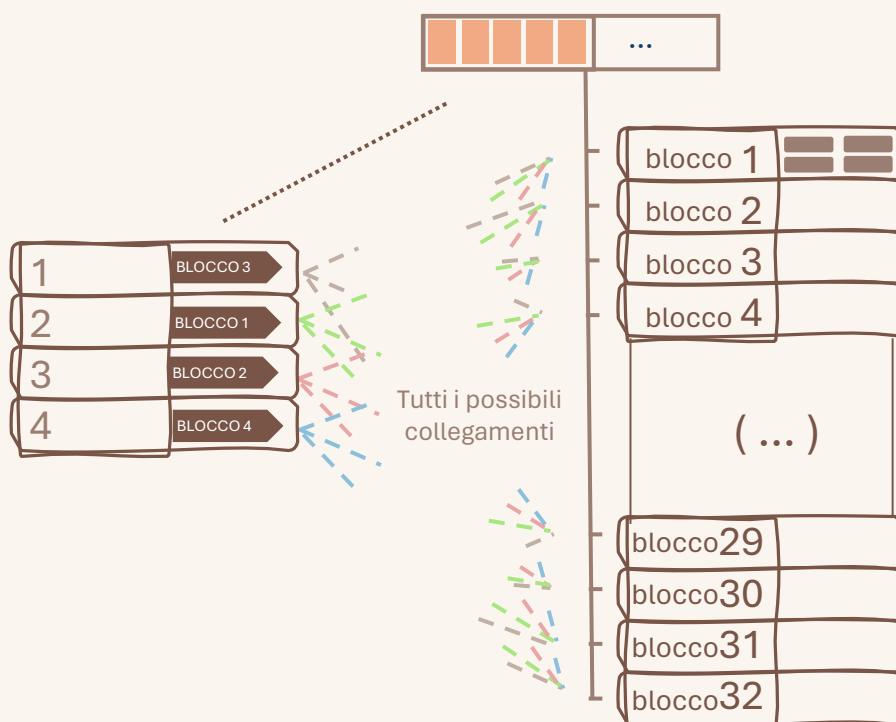
Ogni blocco può stare ovunque

Questo tipo di indirizzamento è più costoso perché per stabilire se la parola di interesse sia presente bisogna esaminare tutte le etichette di posizione registrate confrontandole con il campo etichetta di indirizzo della parola, cercando il blocco/ la pagina a cui la parola appartiene. I conflitti si generano solamente se la CACHE è piena.



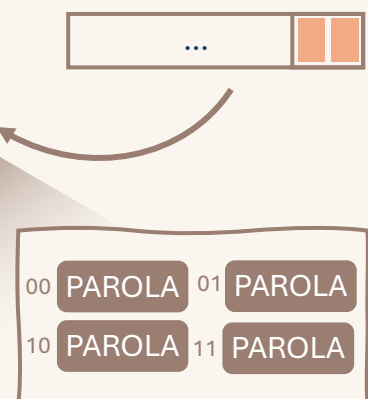
CAMPO DI INDIRIZZO ETICHETTA

occupa 5 BIT necessari per identificare quale tra i 32 blocchi



CAMPO SPIAZZAMENTO

occupa 2 BIT necessari per identificare quale tra le 4 parole possibili



COME ASSOCIARE UN BLOCCO IN MEMORIA AD UNA POSIZIONE IN CACHE

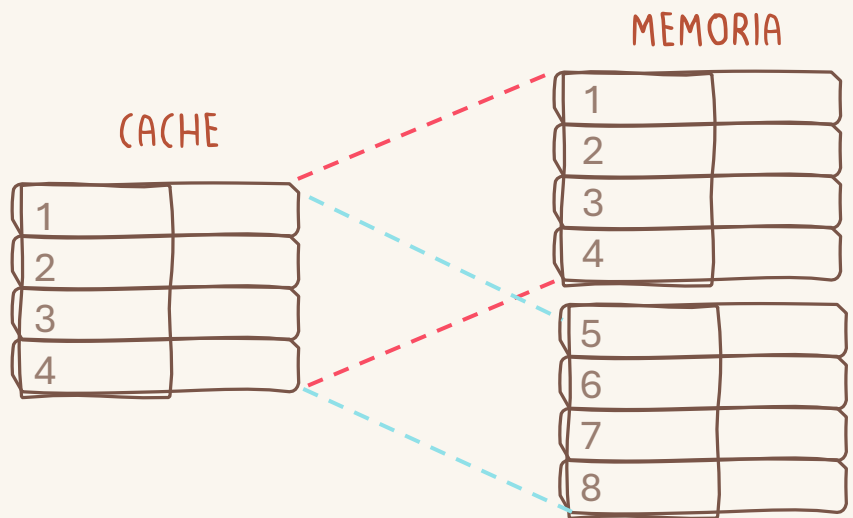
METODI DI INDIRIZZAMENTO

DIRETTO

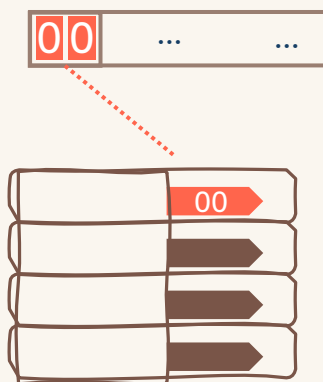
I blocchi 1,2.. Fino a 4 andranno in 1,2,3,4, siccome più blocchi sono caricabili nella medesima posizione, si può avere conflitto quando la posizione è occupata. Il conflitto è risolvibile **sovrascrivendo** il contenuto della posizione, i blocchi 5,6,7.. andranno a **sovrascrivere** i blocchi 1,2,3.. E così via, si sovrascrive tramite i metodi di hit o miss, il blocco che ha in comune gli stessi bit di posizione

L'indirizzo è suddiviso in 3 campi:

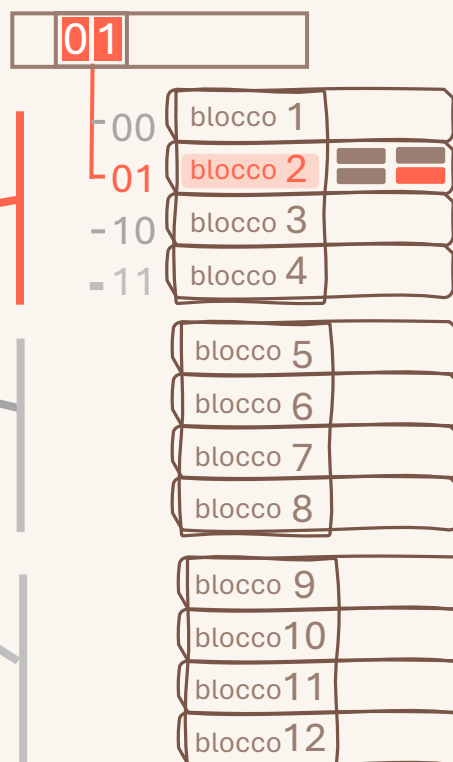
- Il **campo spiazzamento/OFFSET** sono i 3 BIT meno significativi dell'indirizzo e rappresenta lo spiazzamento che individua la parola all'interno del blocco di 4 parole
- Il **campo blocco/PAGINA** sono i 2 BIT centrali rappresenta il numero della posizione (che è unica) cui il blocco è destinato tra le 3 presenti
- Il **campo etichetta**, contenente i 5 BIT più significativi, viene registrato come etichetta associata alla posizione dove è stato caricato il blocco. L'etichetta identifica uno tra i 12 blocchi/4 posizioni = 3 blocchi caricabili nella posizione.



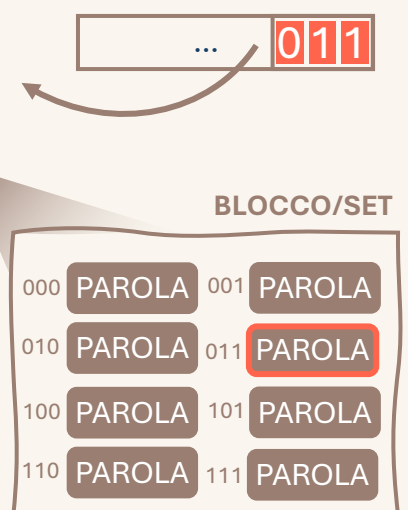
CAMPO ETICHETTA



CAMPO BLOCCO



CAMPO SPIAZZAMENTO



2 2 3

Indirizzo blocco della memoria associato in una posizione nella cache

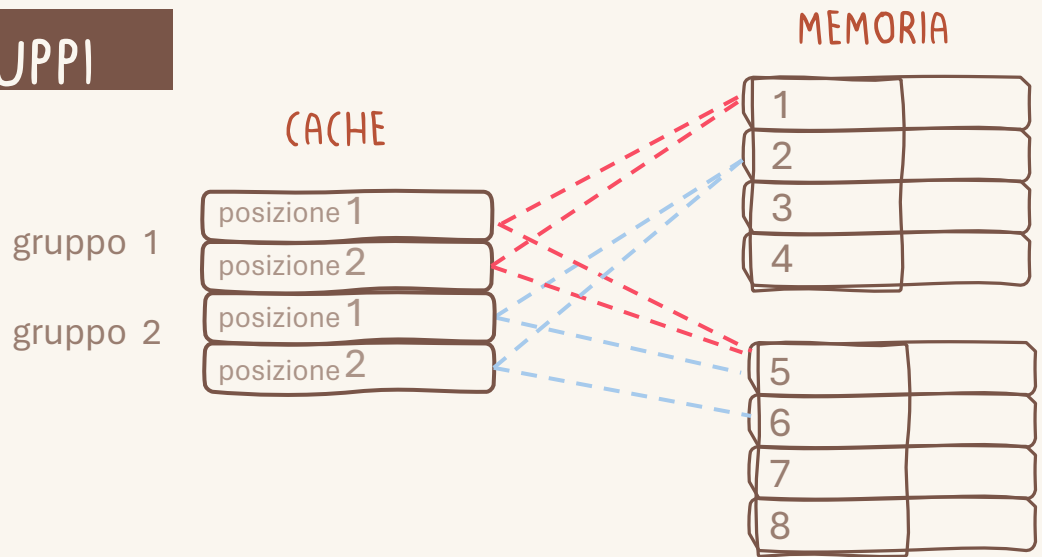
00 01 111

COME ASSOCIARE UN BLOCCO IN MEMORIA AD UNA POSIZIONE IN CACHE

METODI DI INDIRIZZAMENTO

ASSOCIATIVO A GRUPPI

La CACHE ha dei gruppi in questo caso dispone di 2 gruppi, i blocchi **numero 1, 5, 9.. sono caricabili nel gruppo 1** (cioè in una delle due posizioni del gruppo), **i blocchi 2, 6, 10.. sono caricabili nel gruppo 2** e così via, in modo simile alla CACHE diretta, Il campo etichetta identifica il numero del blocco a cui la parola appartiene.



Esso va confrontato, mediante ricerca associativa, con le etichette di posizione registrate in CACHE e appartenenti al gruppo sotto esame, e la posizione in cui ci fosse coincidenza sarebbe quella dove si troverebbe la copia del blocco (cache hit) ; altrimenti il blocco non starebbe in CACHE (cache miss)



CAMPO SPIAZZAMENTO

CAMPO DI GRUPPO



il CAMPO GRUPPO in questo caso è di 2 bit essendoci fino a 4 gruppi

CAMPO DI INDIRIZZO ETICHETTA



il CAMPO ETICHETTA è 2 bit, 12 blocchi/ 4 gruppi = 4 blocchi, bastano 2 bit per rappresentarli



ALGORITMO DI SOSTITUZIONE

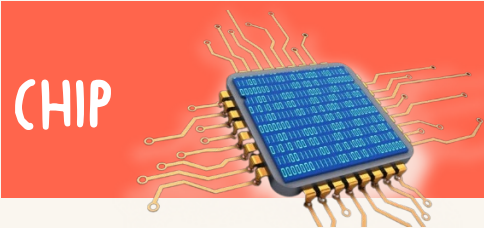
FUNZIONA SOLO PER METODO ASSOCIATIVO A GRUPPI O NON

LRU

LIST RECENTLY USED

Per ogni posizione in CACHE in uno specifico gruppo mantengo un contatore, quando devo sostituire un blocco in una posizione del gruppo, sostituisco quello con il valore di contatore più alto. Il contatore si aggiorna ad ogni hit o miss del gruppo. Ogni contatore ha un valore diverso e crescente. Ad ogni accesso a posizione resetto il contatore riportandolo a zero perche vuol dire che ho usato recentemente quella parola e mi potrebbe riservire. Esistono anche altri algoritmi di sostituzione come FIFO: che elimina il più vecchio e CASUALE: che elimina a caso.

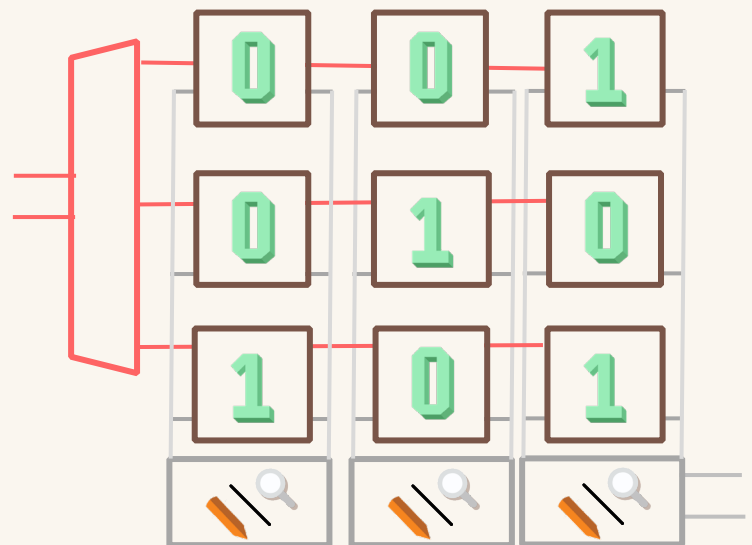
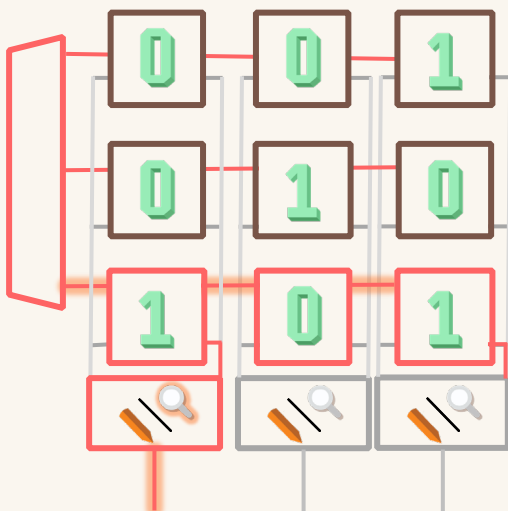
1	0
2	1
3	0
4	3



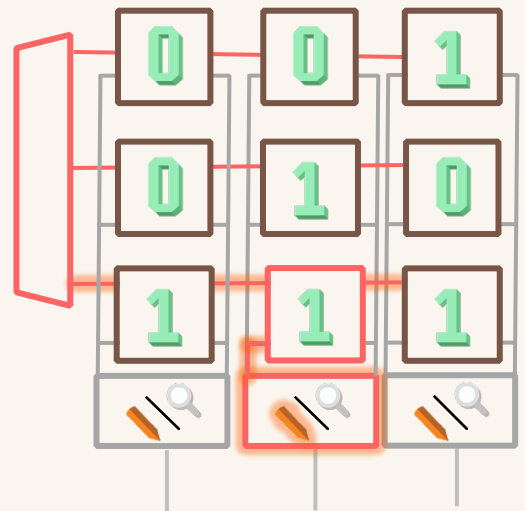
la cella elementare di memoria ha capacità di un bit, solitamente sono disposte in una matrice. Una riga celle da un bit sono tutte collegate in fila mediante una linea apposita comandata dal decodificatore di indirizzo detta word line, ciascuna colonna invece è collegata al dispositivo di lettura/scrittura tramite due linee

Ad esempio se una matrice ha 16 parole da 8 bit ciascuna, si può esprimere ciò nella notazione 16x8, che sono 128 bit, esternamente si hanno dei collegamenti andare all'interno: 4 di indirizzo, 8 di dato, 1 di R/W (lettura/scrittura negata), 2 di alimentazione (massa e vcc), 1 di CS (chip select).

IN LETTURA il circuito rileva e amplifica il valore elettrico contenuto nella cella interessata e lo manda al morsetto corrispondente di dato su cui esce come bit.



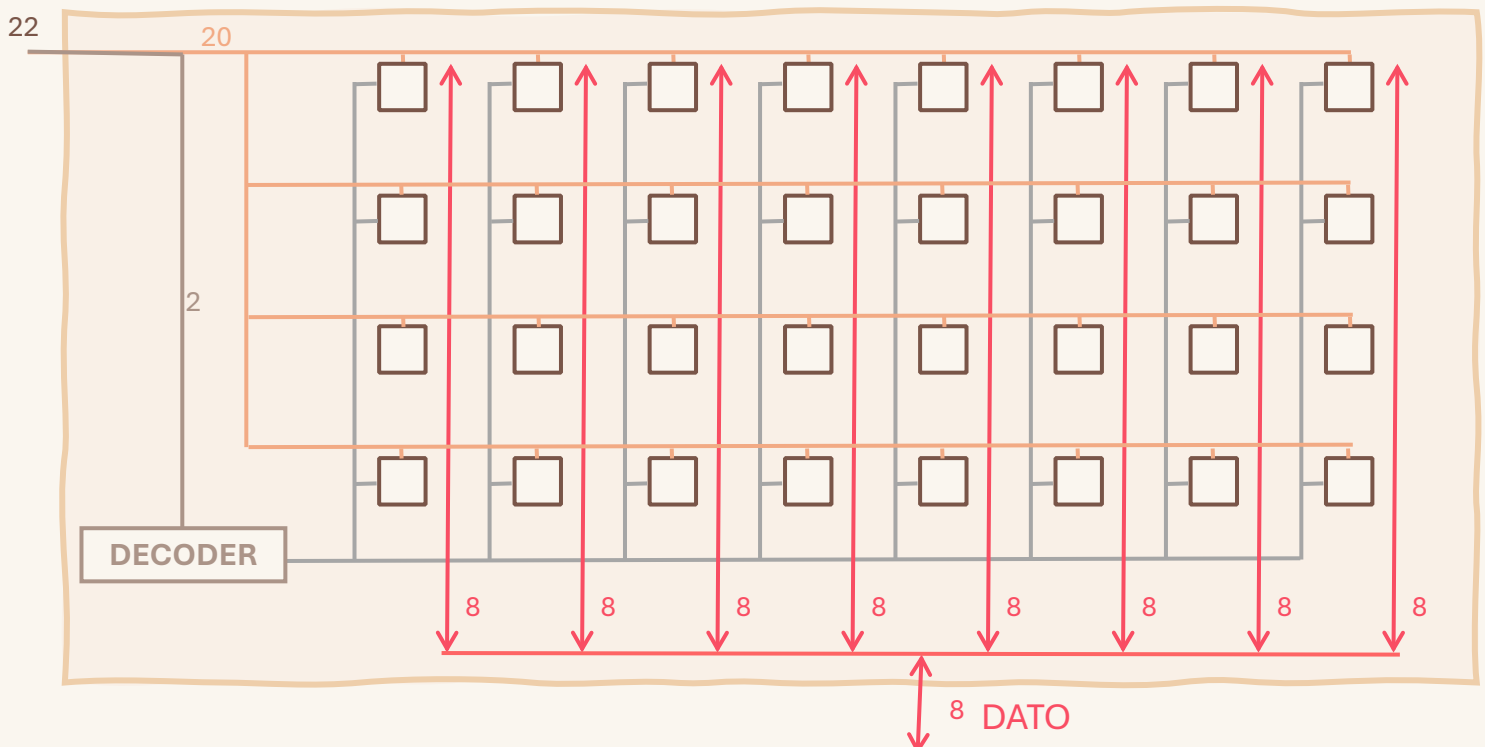
IN SCRITTURA avviene il processo contrario, si rileva la tensione presente sul morsetto e la si va ad applicare alla cella corretta



Si realizzi un banco di memoria statica di 4MB indirizzabile al byte utilizzando chip 1M x 1 bit .

Dobbiamo creare una (SRAM) di dimensioni totali di 4MegaByte (1 MB = 2^{20} byte, 1 byte = 8bit), composta da componenti chip 2^{20} bit x 1 bit.

Bisogna stare attenti a non confondere MB e M, per creare una sram di 4MB basta avere 4 e 8 chip per lato(4M x 8bit) , 20bit ci serviranno per l'indirizzamento dentro al chip e ultimi bit per indirizzare gruppi da 1 byte, essendoci 4 gruppi bastano 2 bit, l'indirizzo totale è 22 bit, puoi avere conferma di cio facendo $\log(\text{memoria}/\text{unità indirizzabile}) = \log(4\text{MB}/1\text{b})=22$



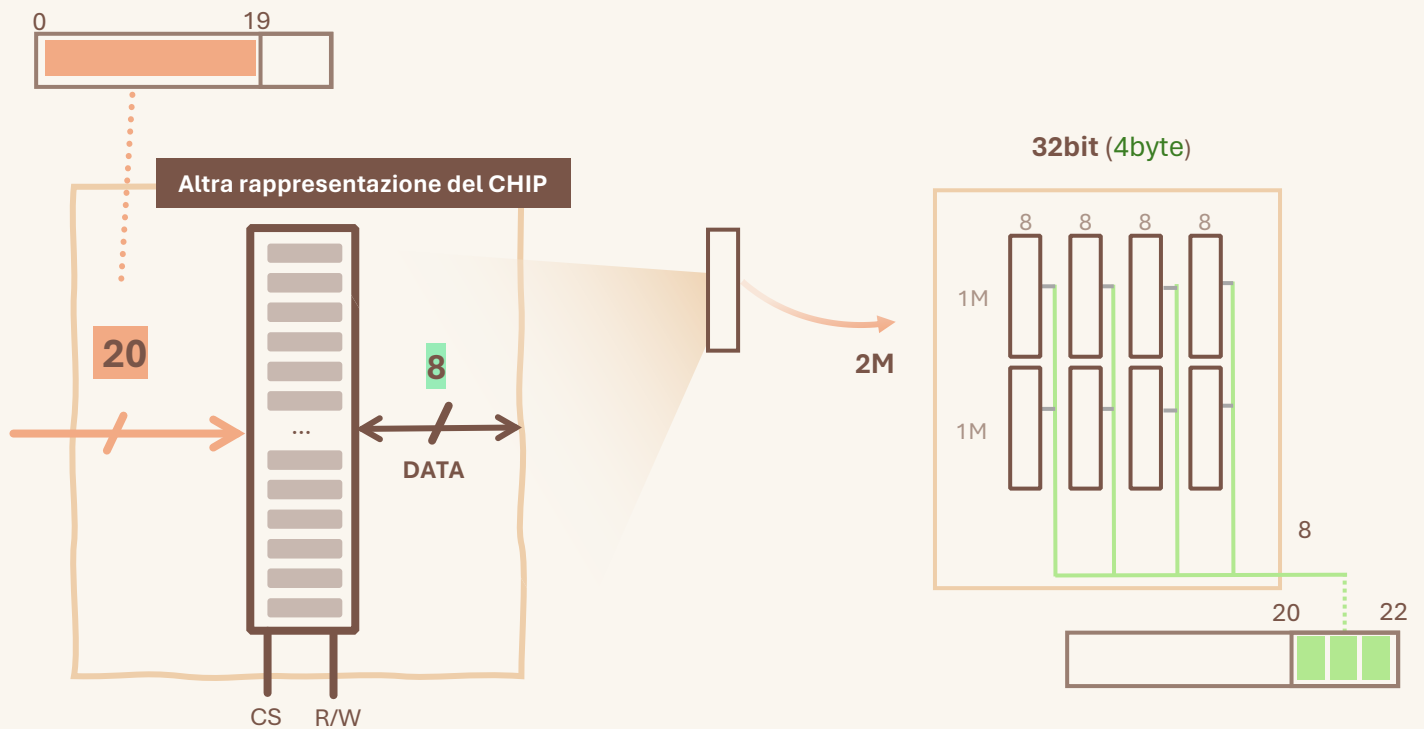
BANCHI DI MEMORIA ESERCIZI

Esempio:

Si realizzi una SRAM di $2\text{MB} \times 32\text{ bit}$ indirizzabile al byte utilizzando chip $1\text{M} \times 8\text{ bit}$.

Spiegazione:

Dobbiamo creare una memoria di dimensioni $2^{21} \times 32\text{ bit}$, composta da chip $2^{20} \times 1\text{ bit}$. Per creare una sram di queste dimensioni basta avere 2 e 4 chip per lato ($2\text{M} \times 4\text{ byte} = 2\text{M} \times 32\text{bit}$), avendo chip da 8 bit ovvero 1 byte, bisogna poterli indirizzare tutti a 8 con 3 bit



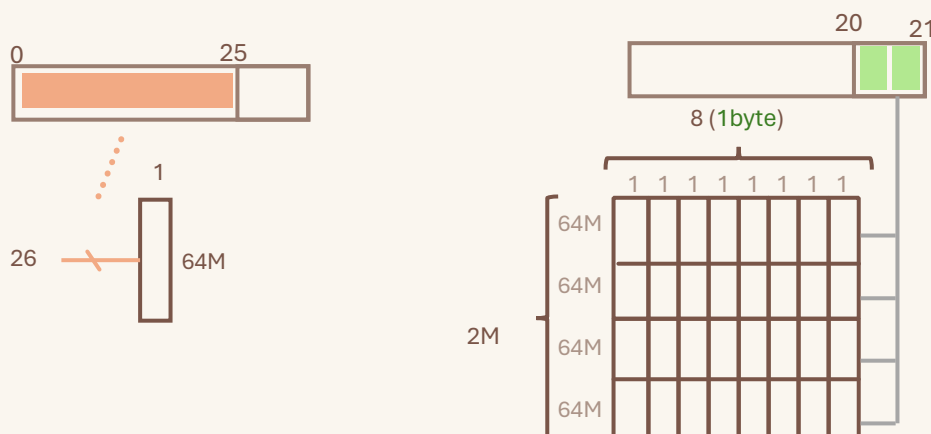
$$2\text{M} \times 4\text{ byte} = 2\text{M} \times 4 (8\text{bit}) = 2\text{M} \times 32\text{ bit}$$

Esempio:

Si realizzi una DRAM di $256\text{M} \times 8\text{ bit}$ indirizzabile al byte utilizzando chip $64\text{M} \times 1\text{ bit}$.

Spiegazione:

Per creare una memoria con queste dimensioni composta con questi chip, bisogna creare un rettangolo con 4 e 8 chip per lato, infatti $256\text{M}/64\text{M} = 4$ e $8\text{bit}/1\text{bit} = 8$, l'esercizio chiede di poter indirizzare parole da 1 byte (8bit) essendo che da ogni chip esce un 1 bit c'è ne servono 8 per volta, poi per indirizzarli tutti ci servono 2 bit essendo che ci sono 4 gruppi, $((4 \times 8)/8 = 4\text{ gruppi})$



MICROISTRUZIONE ESERCIZIO

Esempio:

CALL (EAX)

1. PC_{OUT} , MAR_{IN} , READ, SELECT[4], ADD, Z_{IN}
2. WMFC, Z_{OUT} , PC_{IN}
3. MDR_{OUT} , IR_{IN}

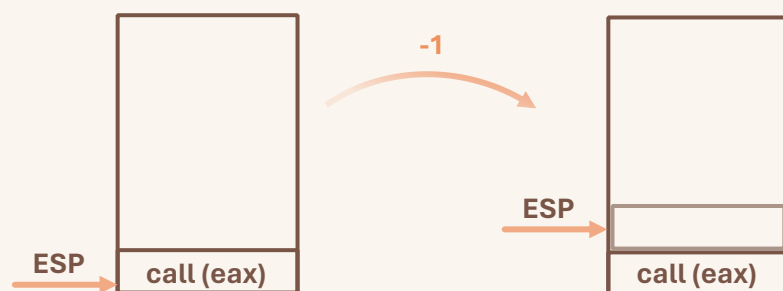
Fetch

4. ESP_{OUT} , SELECT[4], SUB, Z_{IN}
5. Z_{OUT} , ESP_{IN} , MAR_{IN}
6. PC_{OUT} , MDR_{IN} , WRITE
7. WMFC

Salvataggio nello stack (PUSH)

Salvataggio dello stack:

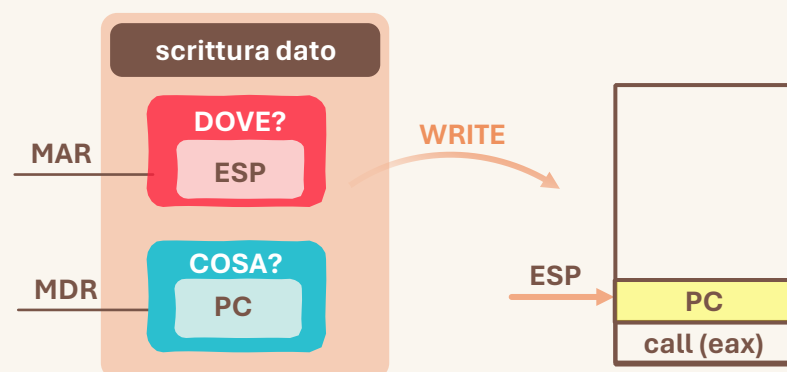
4. Viene decrementato il valore dell'ESP (stack pointer) per puntare alla nuova cima dello Stack



5. Il nuovo valore viene caricato in ESP e nel MAR per la fase di scrittura

6. Il PC passa nel registro dei dati MDR e viene eseguita l'operazione di scrittura, salvando in memoria (nella cima dello Stack) l'indirizzo attuale di PC.

7. Attesa scrittura



Chiamata a funzione:

8. Il valore di EAX viene passato al MAR come indirizzo di memoria. Viene comandata una lettura.

9. Attesa lettura; nel frattempo PC viene caricato nel registro V, siccome il BUS non è occupato.

10. Dal MDR esce il valore letto che viene sommato a V in quanto il salto è relativo.

11. L'indirizzo risultante dell'istruzione da eseguire viene passato al PC. Termine dell'istruzione.

8. EAX_{OUT} , MAR_{IN} , READ

9. WMFC, PC_{OUT} , V_{IN}

10. MDR_{OUT} , SELECT[V], ADD, Z_{IN}

11. Z_{OUT} , PC_{IN} , END

